

Reasoning Patterns for Efficient AI Coding: An Empirical Study

ANONYMOUS AUTHOR(S)

Long reasoning traces generated by Large Reasoning Models (LRMs) provide a rich behavioral record for understanding and controlling model inference, yet the underlying reasoning patterns and how different patterns affect coding models remain poorly understood, limiting systematic characterization and effective utilization of model reasoning behavior.

We conduct an empirical study of reasoning patterns across four coding tasks. We transform reasoning traces into structured action sequences and mine recurring fragments using within-task support, cross-task contrasts, and valid-invalid support contrasts[gu: how do we mine patterns?][yin: done.]. We characterize four representative general reasoning patterns[gu: XXX][yin: done.]: design-to-implementation for code generation, state tracing for execution reasoning, evidence-fix-recheck for debugging, and semantic mapping for translation. We further identify positive and negative patterns associated with valid and invalid reasoning traces, respectively. Positive patterns reflect verification-driven convergence, whereas negative patterns reflect repeated design revision, reflection, and knowledge recall without closure.

To demonstrate their practical utility, we evaluate three pattern-based applications: task identification, pattern-guided prompting, and pattern-aware early stopping[gu: ??][yin: done.]. Features capturing general reasoning patterns enable accurate task identification from reasoning traces without accessing task descriptions or final answers[gu: not clear][yin: done.]. Prompting guided by positive and negative patterns improves the overall accuracy of downstream tasks, while negative pattern-aware early stopping substantially reduces API-reported token usage with only a modest decrease in pass@1. All three applications also generalize to previously unseen models, demonstrating the utility of reasoning patterns for both analyzing and controlling LRM behavior in AI coding.

Additional Key Words and Phrases: reasoning patterns, AI coding, reasoning trace analysis, reasoning pattern mining, inference-time optimization

ACM Reference Format:

Anonymous Author(s). 2026. Reasoning Patterns for Efficient AI Coding: An Empirical Study . 1, 1 (October 2026), 21 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Large reasoning models (LRMs) are rapidly reshaping AI coding [35, 36]. Unlike conventional code language models that generate an answer directly, LRMs produce a sequence of intermediate reasoning steps before arriving at a solution. This capability is particularly valuable for coding tasks, where solving a problem often involves intermediate decisions, such as decomposing requirements and designing an algorithm for code generation or tracking program states during execution reasoning. The reasoning process is therefore exposed as part of the model’s output rather than remaining implicit.

The availability of these reasoning traces has led to a growing body of research on their role in code-related tasks [6, 29]. Existing studies show that intermediate reasoning can improve task

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/10-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

performance, but also reveal substantial inefficiencies. Reasoning traces may contain incorrect assumptions, redundant exploration, repeated revisions, and prolonged deliberation that adds inference cost without improving the final result [4, 12, 18, 39]. These observations have motivated the use of reasoning traces for purposes beyond generating an answer, including self-correction [20, 22], evaluation [26, 30], and inference-time control [9, 10].

However, most existing studies examine reasoning from the perspective of individual tasks or specific behaviors. They focus on final task performance [3, 15, 16], prompting strategies [13, 27, 29], or particular reasoning behaviors in tasks such as code generation [12, 18, 31, 39]. This leaves a more basic question largely unanswered: do reasoning traces contain recurring structures that characterize how LRMs solve coding tasks? If such structures exist, it is also unclear which of them are associated with successful or unsuccessful reasoning, and whether they can be used to improve the efficiency and reliability of inference. Addressing these questions requires looking beyond individual reasoning traces and examining them as sequences of recurring reasoning actions.

To address these questions, we conduct an empirical study of reasoning patterns in AI coding. We collect reasoning traces generated by LRMs across four representative coding tasks: code generation, code execution reasoning, program debugging, and code translation. We transform these traces into structured reasoning action sequences using a unified action taxonomy. This representation allows us to analyze reasoning in terms of observable actions rather than the surface form of generated text. We then examine reasoning patterns at two levels. General patterns capture recurring task-level reasoning structures, whereas positive and negative patterns capture reasoning behaviors at the action level that distinguish valid from invalid trajectories. Finally, we investigate whether the discovered patterns can be leveraged for inference-time analysis and control of LRM behavior.

Specifically, we investigate the following research questions:

- **RQ1 (General Patterns): What task-level reasoning patterns emerge across different coding tasks?**

Different coding tasks impose distinct reasoning demands, yet it remains unclear whether these differences are reflected in recurring task-level structures. To characterize such structures, we represent reasoning traces as action sequences using a unified action taxonomy and mine recurring action fragments across the four coding tasks. We then examine the resulting patterns to identify the reasoning strategies that characterize each task.

- **RQ2 (Positive and Negative Patterns): What action-level reasoning patterns distinguish valid from invalid reasoning trajectories?**

General patterns capture differences across coding tasks, but they do not reveal the local reasoning behaviors underlying trace validity. We therefore compare valid and invalid reasoning traces and mine recurring action fragments that characterize the two groups. For tasks with objective correctness signals, we assess trace validity using an ensemble LLM-as-a-judge protocol. We then identify positive patterns prevalent in valid reasoning and negative patterns prevalent in invalid reasoning.

- **RQ3 (Applications): Can the discovered reasoning patterns be leveraged to analyze and control LRM inference?**

We further examine whether the discovered patterns can be used to guide and control inference. Specifically, we evaluate three applications. First, we use general pattern features to identify coding tasks from reasoning traces. Second, we augment prompts with guidance derived from positive and negative patterns to steer reasoning behavior. Third, we use negative pattern-aware early stopping to terminate inference when such patterns are detected and examine its effects on coding accuracy and token usage.

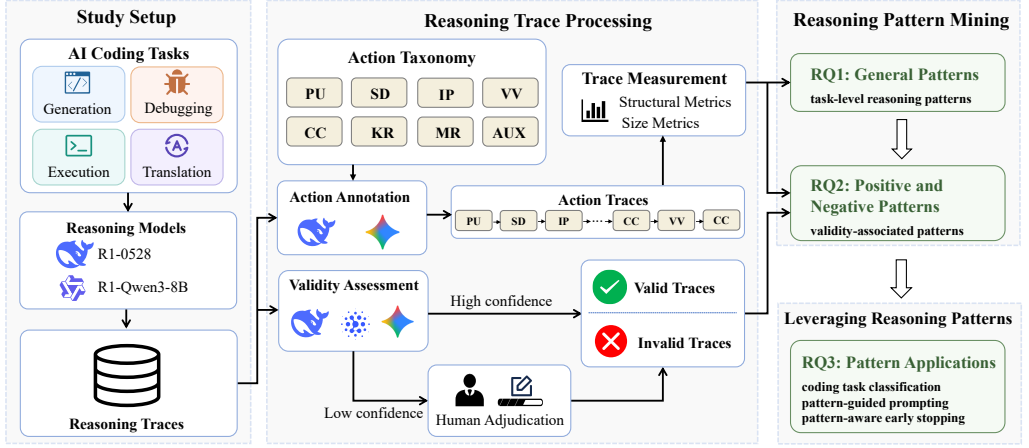


Fig. 1. Overview of the Empirical Study.

Our study yields three main findings. At the task level, coding tasks exhibit distinct general reasoning structures: code generation follows a design-to-implementation flow, code execution reasoning centers on state tracing, debugging follows an evidence-fix-verification loop, and code translation centers on semantic mapping. At the action level, valid and invalid reasoning trajectories exhibit distinct positive and negative patterns. Valid traces show verification-driven convergence, whereas invalid traces exhibit recurring behaviors such as repeated redesign, reflection, and knowledge recall without resolution. Beyond characterization, the discovered patterns support three inference-time applications across the evaluated task-model settings. General patterns support task identification, prompting guided by positive and negative patterns improves single-shot correctness, and negative pattern-aware early stopping substantially reduces API-reported token usage while incurring only a modest decrease in pass@1. All three applications also generalize to previously unseen models.

In summary, this paper makes the following contributions:

- We establish a unified reasoning-action representation for cross-task analysis of LRM reasoning traces and conduct an empirical study across four representative AI coding tasks.
- We characterize reasoning in AI coding at two levels, identifying general patterns at the task level and positive and negative patterns at the action level from valid and invalid reasoning trajectories, respectively.
- We demonstrate the utility of these patterns for AI coding through three inference-time applications: task identification using general patterns, prompting guided by positive and negative patterns, and negative pattern-aware early stopping.

2 Methodology

Figure 1 presents an overview of our empirical study. First, we collect reasoning traces generated by two LRMs across four representative AI coding tasks and assess the validity of each trace. Second, we convert each trace into a sequence of reasoning actions using a unified reasoning-action taxonomy and quantify the resulting behaviors through size and structural metrics. Third, we mine recurring patterns from the resulting action traces, including task-level general patterns for RQ1

and action-level positive and negative patterns for RQ2. Finally, in RQ3, we evaluate how the discovered patterns can be leveraged for AI coding through three inference-time applications: coding task identification using general patterns, prompting guided by positive and negative patterns, and negative pattern-aware early stopping.

2.1 Study Setup

This section describes the coding tasks, datasets, and LRMs used in our empirical study.

2.1.1 Tasks and Datasets. To study reasoning patterns in AI coding across diverse problem-solving settings, we consider four representative coding tasks covering program synthesis, comprehension, maintenance, and transformation:

- *Code generation* requires a model to synthesize executable programs from natural-language specifications. We use LiveCodeBench v4–v6 [15], comprising 442 instances.
- *Code execution reasoning* requires a model to infer the output of a program through static reasoning rather than actual execution. We use the input–output prediction subset of CodeSense [21], comprising 382 instances.
- *Program debugging* requires a model to localize and repair defects in buggy programs. We use DebugBench [25], comprising 500 instances.
- *Code translation* requires a model to transform programs across programming languages while preserving their semantics. We use ClassEval-T [32], comprising 376 instances.

All four tasks provide objective correctness oracles, which allow us to analyze both reasoning traces and task outcomes under a unified experimental setting.

2.1.2 Models. For primary pattern discovery, we use two publicly available LRMs, DeepSeek-R1-0528 (denoted as R1-0528) and DeepSeek-R1-0528-Qwen3-8B (denoted as R1-Qwen3-8B) [6]. R1-Qwen3-8B is obtained by distilling R1-0528 into a Qwen3-8B backbone. For every task instance, both models are evaluated using identical prompts and decoding configurations. We collect the complete reasoning trace and final response produced by each model, resulting in the dataset used for subsequent trace processing and pattern analysis.

For pattern utilization, ****. [shen: complete this para.]

2.2 Reasoning Trace Processing

This section describes how we process the collected reasoning traces into reasoning action traces and the derived metrics used for subsequent reasoning pattern mining and analysis.

2.2.1 Reasoning Trace Collection. For each task instance, we instantiate the benchmark-provided prompt template with the corresponding instance-specific input, such as a problem description, starter code, buggy program, or source-language snippet. We submit each instantiated prompt independently to both models in thinking mode under a unified decoding configuration (temperature = 0.6, top- p = 0.95).

We extract the content enclosed by the <think> tags as the reasoning trace. To ensure completeness, we regenerate any response with an empty reasoning trace or lacking the information required for annotation or task evaluation, until a complete trace and task output are obtained.

After excluding incomplete, unparsable, or unlabeled responses, the resulting discovery dataset contains 3,376 reasoning traces across the four tasks and two models. Specifically, we retain 442/440 traces for code generation, 377/378 for code execution reasoning, 499/496 for program debugging, and 372/372 for code translation on R1-0528/R1-Qwen3-8B, respectively.

2.2.2 Reasoning Trace Validity Assessment. To distinguish valid from invalid reasoning behaviors in RQ2, we assess the validity of each reasoning trace using an ensemble LLM-as-a-judge protocol [17, 40]. Trace-level validity is distinct from task-level functional correctness: a trace may lead to a correct final answer while containing logical inconsistencies, unsupported transitions, or unnecessary reasoning detours. We consider a reasoning trace *valid* if its reasoning is internally consistent and progresses coherently toward the final solution.

We employ three judge models, DeepSeek-V3.2 [7], Gemini-3-Flash [11], and GLM-5.1 [37]. We use a unified evaluation prompt that provides the same validity criteria and requires each judge to independently classify a trace as *valid* or *invalid* and report a confidence score in the range [0, 1]. To reduce the effect of stochastic judgment variation, we query each judge three times for each trace, yielding nine judgments per trace.

We aggregate the nine judgments using confidence-weighted voting. For a candidate label ℓ , its normalized support is defined as:

$$S(\ell) = \frac{\sum_{m \in \mathcal{M}} \sum_{r=1}^3 c_{m,r} \cdot \mathbf{1}[y_{m,r} = \ell]}{\sum_{m \in \mathcal{M}} \sum_{r=1}^3 c_{m,r}}. \quad (1)$$

We assign the label with the larger normalized support and use its support as the aggregate confidence. Traces with an aggregate confidence above 0.8 are automatically assigned the corresponding label; the remaining traces are adjudicated independently by two human annotators. The resulting labels define the valid and invalid reasoning-trace groups used in RQ2. The final dataset contains 2,452 valid and 924 invalid traces. R1-0528 contributes 1,439 valid and 251 invalid traces, while R1-Qwen3-8B contributes 1,013 valid and 673 invalid traces.

2.2.3 Reasoning Action Taxonomy and Annotation. Reasoning traces are free-form natural-language text, making systematic comparison across tasks and models difficult. To enable structured analysis, we represent each reasoning trace as a sequence of fine-grained *reasoning actions*, where a reasoning action is defined as a contiguous text span expressing a distinct reasoning behavior.

Drawing on prior studies of LLM reasoning [12, 20, 22, 35], we define a taxonomy of eight reasoning actions that captures the major stages of code reasoning, ranging from task understanding and solution construction to verification and reflection. To strengthen its validity, the taxonomy is further inspected by two researchers with over five years of experience in code intelligence systems, who independently evaluated the coverage, granularity, and ambiguity of the action definitions and iteratively refined them to resolve overlaps. The resulting taxonomy provides a task-independent representation of reasoning behaviors across different coding tasks. Table 1 summarizes the eight reasoning actions.

Using this taxonomy, we automatically annotate each reasoning trace with DeepSeek-V3.2 [7] and Gemini-2.5 Pro [24]. Specifically, each reasoning trace is segmented into contiguous spans, and each span is assigned exactly one reasoning action. To ensure consistent annotations, we require the models to generate structured JSON outputs containing the assigned reasoning actions, corresponding text spans, and their ordering. The annotation prompt specifies (1) the definitions of the eight reasoning actions with representative examples, (2) segmentation guidelines covering boundary criteria, granularity, and special-case handling, (3) priority rules for resolving ambiguous cases, and (4) the required JSON output schema. The complete annotation prompt is included in our replication package.

To assess annotation reliability, two human annotators independently re-annotated a randomly sampled subset of reasoning traces. Their annotations achieved substantial agreement with the

Table 1. Taxonomy of Reasoning Actions

Abbr.	Reasoning Action	Description
PU	Problem understanding	Understanding and clarifying the problem requirements.
SD	Solution design	Designing the high-level solution approach and strategy.
IP	Implementation planning	Planning the detailed implementation steps and logic.
CC	Code construction	Writing and refining the actual code.
VV	Verification and validation	Testing, debugging, and validating the solution.
KR	Knowledge retrieval	Retrieving and applying relevant knowledge.
MR	Meta-reasoning	Reflecting on and monitoring the reasoning process itself.
AUX	Auxiliary	Auxiliary statements that support the reasoning flow.

LLM-generated annotations (Cohen’s $\kappa = 0.816$) [5], indicating that the proposed taxonomy and annotation protocol produce reliable reasoning-action annotations.

2.2.4 Action Trace Construction. The annotated reasoning actions are converted into an *action trace*, defined as an ordered sequence of reasoning actions:

$$\tau = (s_1, s_2, \dots, s_n),$$

where each $s_i \in \mathcal{C} = \{PU, SD, IP, CC, VV, KR, MR, AUX\}$ denotes one reasoning action from the taxonomy, forming the canonical representation for all subsequent analyses.

To emphasize transitions between reasoning stages while reducing the influence of repeated elaboration within the same action, we further derive a *collapsed action trace* $\bar{\tau}$ by merging adjacent identical actions. For example,

$$(SD, SD, KR, SD, VV, VV) \rightarrow (SD, KR, SD, VV).$$

2.2.5 Action Trace Measurement. We characterize each reasoning action trace using two groups of metrics: *size metrics*, which quantify the length and output volume of a reasoning trace, and *structural metrics*, which capture the distribution and transition dynamics of reasoning actions.

Size metrics quantify the length of the reasoning trace at both the action and token levels. We report the raw action-trace length n and the collapsed action-trace length m , where consecutive repetitions of the same action or action sequence are merged into a single occurrence. Their ratio n/m measures the degree of consecutive action repetition. We additionally compute the number of reasoning tokens generated by the model using a consistent tokenizer across all comparisons. These tokenizer-based counts characterize generated reasoning volume and are distinct from API-reported token usage.

Structural metrics characterize the composition of reasoning actions and their transition dynamics. For each action c , its *allocation* is defined as the fraction of action spans in a trace assigned to c . To characterize reasoning flow, we construct a transition matrix P , where P_{ab} denotes the probability of transitioning from action a to action b . We estimate P_{ab} by counting adjacent $a \rightarrow b$ transitions in collapsed traces and normalizing by the total number of outgoing transitions from a . We then quantify transition diversity using the stationary-distribution-weighted average of the row-wise transition entropies:

$$H(P) = \sum_a \pi_a \left(- \sum_b P_{ab} \log_2 P_{ab} \right), \quad (2)$$

where π denotes the stationary distribution of P . Higher entropy indicates greater diversity in the actions that can follow a given action, reflecting a less concentrated transition structure.

2.3 Reasoning Pattern Mining

This section describes how we mine reasoning patterns from the resulting reasoning action traces and characterize them at two granularities: *general patterns* at the task level and *positive and negative patterns* at the action level, corresponding to RQ1 and RQ2, respectively.

2.3.1 Reasoning Pattern Definition. A *reasoning pattern* is a recurring action-fragment signature identified from a collection of LRM reasoning action traces. Given a trace set $\mathcal{T}$, the *support* of a signature s is the fraction of traces in $\mathcal{T}$ whose collapsed action trace contains s :

$$\text{supp}(s, \mathcal{T}) = \frac{|\{t \in \mathcal{T} : s \subseteq t\}|}{|\mathcal{T}|}. \quad (3)$$

A signature is considered a candidate pattern when its support exceeds a predefined minimum support threshold. We treat reasoning patterns as observable regularities in action traces rather than as direct evidence of the model's latent cognitive mechanisms.

We distinguish two types of reasoning patterns according to their analysis granularity. *General patterns* capture task-level reasoning structures through ordering relations among actions. We denote $A \Rightarrow B$ when action A precedes action B in a trace, allowing arbitrary intermediate actions between them. Thus, this representation captures coarse-grained ordering relations without requiring the actions to be adjacent. *Positive and negative patterns*, in contrast, capture local reasoning structures through adjacent action transitions. We denote $A \rightarrow B$ when A and B occur consecutively, and $A \leftrightarrow B$ when A and B repeatedly alternate over adjacent positions, forming a short cyclic loop. These representations provide the basis for identifying task-level regularities and distinguishing local patterns prevalent in valid and invalid reasoning trajectories, respectively.

2.3.2 Mining General Patterns. General patterns are mined to identify recurring action structures that distinguish one coding task from the others. For each task, we extract candidate action-fragment signatures from its collapsed reasoning traces. We consider three forms of signatures: contiguous action n -grams with lengths of 4–5, non-contiguous ordered action paths, and short cyclic loops. For example, $PU \Rightarrow SD \Rightarrow IP \Rightarrow CC$ denotes an ordered pattern in which the four actions occur in the specified order, while allowing arbitrary intermediate actions.

We then assess the task specificity of each candidate signature using its within-task support and its support difference from the other tasks. A signature is retained as a general pattern for a task only if its support within that task exceeds the minimum support threshold and its support advantage over the other tasks exceeds a predefined margin. The resulting general patterns capture recurring task-level reasoning structures and are subsequently used for task identification in RQ3.

2.3.3 Mining Positive and Negative Patterns. Positive and negative patterns are mined to identify recurring action-level structures that distinguish valid reasoning traces from invalid ones. We examine the two trace groups from complementary perspectives: aggregate action allocation and local action transitions. The former characterizes how reasoning actions are distributed across a trace, whereas the latter captures recurring local structures in the action sequence.

The first perspective focuses on *action allocation*. We define the *closure share* as the combined proportion of action spans labeled verification and validation (VV), code construction (CC), or auxiliary (AUX). This aggregate measure captures the extent to which a trace allocates reasoning toward convergence-oriented actions. Here, *AUX* retains its taxonomy definition of auxiliary statements and does not specifically denote answer finalization. Conversely, the *drift share* is the combined proportion of spans labeled solution design, meta-reasoning, or knowledge retrieval, capturing allocation toward actions that revisit, extend, or redirect the reasoning process. We summarize the relative balance between the two using the *closure-minus-drift* score, defined as

the closure share minus the drift share. Positive values indicate greater allocation toward closure-oriented actions, whereas negative values indicate greater allocation toward drift-oriented actions.

The second perspective focuses on *local structural patterns*. From collapsed action traces, we extract contiguous action n -grams and represent their local structures using adjacent transitions $A \rightarrow B$ and short alternating loops $A \leftrightarrow B$. These patterns capture recurring action-level structures that are not reflected in aggregate allocation measures. Whereas closure and drift shares characterize the overall allocation of reasoning actions, local patterns identify the specific action transitions and short sequences through which these allocation tendencies manifest.

For local patterns, we identify group-enriched candidates based on both support and support contrast. A candidate pattern must exceed a minimum support threshold in its enriched group and exhibit a minimum support difference relative to the comparison group. Patterns more prevalent in valid traces are labeled *positive*, whereas those more prevalent in invalid traces are labeled *negative*. We assess the statistical significance of the resulting group differences using two-sided Fisher’s exact tests with Benjamini–Hochberg correction [2].

2.4 Leveraging Reasoning Patterns

This section describes how we leverage the discovered reasoning patterns for three inference-time applications in RQ3: coding task identification using general patterns, pattern-guided prompting using positive and negative patterns, and negative pattern-aware early stopping.

2.4.1 Coding Task Identification. We train a supervised probe on features extracted from reasoning action traces to identify the coding task performed by an LRM. The features operationalize the general patterns identified in RQ1 by encoding how a trace allocates actions, transitions between them, and distributes them across reasoning stages. We construct 57 features in seven groups: (1) eight action-allocation features; (2) two trace-size features; (3) twelve transition-probability features; (4) three repetition features; (5) twenty-four position-aware action-allocation features; (6) four action-effort features; and (7) four aggregate features. The probe uses only reasoning-trace features and has no access to task descriptions, final answers, or external metadata.

2.4.2 Pattern-Guided Prompting. We augment task prompts with guidance derived from the positive and negative patterns identified in RQ2. Positive patterns are presented as desirable reasoning structures, while negative patterns are expressed as constraints to discourage recurring undesirable loops, such as $SD \rightarrow KR \rightarrow SD$ and $VV \rightarrow SD \rightarrow MR$. The original task instruction is preserved, with pattern-derived guidance added as inference-time control.

2.4.3 Negative Pattern-aware Early Stopping. We further use negative patterns as online signals to terminate unproductive reasoning before the model completes its original generation. Because full-trace LLM annotation is unsuitable for online intervention, we employ a lightweight rule-based action mapper separate from the LLM-based annotation used for pattern discovery in RQ1 and RQ2. The mapper processes streamed reasoning_content, segments it into micro-units, and assigns each unit one of eight fine-grained actions and one of four coarse states (Explore, Build, Verify, or Wrap) using lexical cues. Consecutive units are grouped into segments based on coarse-state changes, fine-action transitions, and lexical novelty, and each segment receives its majority action label. Adjacent segments with identical labels are collapsed before matching task-specific motifs. The mapper operates locally without additional model or API calls.

At each completed segment, the stopping policy combines negative motifs with recent exploration, lexical novelty, and build/verify history. A loop trigger requires sustained drift and insufficient closure evidence, subject to task-specific minimum-progress guards, while a separate closure trigger requires strong closure evidence and answer or wrap indicators. A repeated-text detector

Table 2. General Reasoning Patterns Identified across Four Coding Tasks

Task	General Pattern	Representative Signature	Support (within-task vs. cross-task)	
			R1-0528	R1-Qwen3-8B
Generation	Design-to-implementation	$PU \Rightarrow SD \Rightarrow IP \Rightarrow VV$	91.4% vs. 17.0%	88.0% vs. 22.0%
Execution Reasoning	State tracing	$PU \Rightarrow \{VV, MR\}^*$	45.9% vs. 2.2%	53.5% vs. 3.4%
Debugging	Evidence-fix-recheck	$\{SD, IP\} \Rightarrow CC \Rightarrow VV$	68.1% vs. 21.5%	70.8% vs. 20.4%
Translation	Semantic mapping-to-implementation	$PU \Rightarrow SD \Rightarrow IP \Rightarrow CC$	36.5% vs. 9.5%	30.1% vs. 16.2%

provides an additional safeguard against degenerate output. When a stopping condition is met, generation is terminated and the original task with the generated reasoning prefix is submitted to the same model in a second request for finalization. The request uses answer-only instructions and disables thinking where supported; any reasoning generated during finalization remains included in the API-reported token usage.

3 RQ1: General Patterns

3.1 Setup

RQ1 characterizes task-level reasoning behaviors using the general pattern mining procedure described in Section 2.3.2. We analyze 3,376 collapsed reasoning action traces generated by two LRMs across four coding tasks. A general pattern is retained if its within-task support exceeds 0.05 and its support difference from the other tasks exceeds 0.03. These thresholds were calibrated by two experts through empirical inspection of preliminary results.

3.2 Results

Across the four coding tasks, we identify a distinct general reasoning pattern for each task. Table 2 reports one representative signature per task and its within- versus cross-task support. The large support gaps (e.g., 91.4% vs. 17.0% for code generation and 45.9% vs. 2.2% for code execution reasoning on R1-0528) indicate that these signatures are substantially more prevalent within their corresponding tasks than across other tasks.

Code Generation Follows a Design-to-Implementation Pattern. Both models progressively move from solution design to implementation planning and verification. R1-0528 interleaves design, planning, and verification ($PU \rightarrow SD \rightarrow VV \rightarrow SD \rightarrow IP \rightarrow VV$), with $IP \leftrightarrow VV$ as the dominant local interaction. R1-Qwen3-8B follows the same structure while incorporating additional meta-reasoning ($PU \rightarrow SD \rightarrow MR \rightarrow SD \rightarrow VV$).

Code Execution Reasoning Follows a State-Tracing Pattern. Both models progressively simulate program states and verify intermediate results. R1-0528 follows a compact trajectory of state tracing and output verification ($PU \rightarrow VV \rightarrow MR$), with the lowest transition entropy across the four tasks (1.690). R1-Qwen3-8B exhibits a longer verification-reflection cycle ($PU \rightarrow VV \rightarrow MR \rightarrow VV$), indicating more iterative transitions within the same overall structure.

Program Debugging Follows an Evidence-Fix-Recheck Pattern. Both models repeatedly alternate between fault validation, code modification, and re-verification. R1-0528 enters verification early and repeatedly alternates between IP and VV ($PU \rightarrow VV \rightarrow IP \rightarrow VV \rightarrow CC \rightarrow VV$), with frequent $PU \rightarrow VV$ and $CC \rightarrow VV$ transitions. R1-Qwen3-8B follows the same structure but more frequently revisits SD after verification ($PU \rightarrow SD \rightarrow VV \rightarrow SD \rightarrow IP \rightarrow CC \rightarrow VV \rightarrow CC$).

Code Translation Follows a Semantic Mapping Pattern. Both models establish semantic correspondences between source and target programs before implementation. R1-0528 transitions

more directly from semantic mapping to implementation and validation ($PU \rightarrow SD \rightarrow IP \rightarrow VV \rightarrow IP$), whereas R1-Qwen3-8B revisits SD more frequently ($PU \rightarrow SD \rightarrow IP \rightarrow SD \rightarrow IP$). The shared signature $PU \Rightarrow SD \Rightarrow IP \Rightarrow CC$ further characterizes translation as semantic reconstruction followed by code construction.

Shared Reasoning Scaffold. Despite their different objectives, the four tasks share an understand-then-act scaffold. PU appears in more than 95% of reasoning traces across both models, after which traces diverge into task-specific sequences involving SD , IP , VV , and CC . VV is prominent in generation, execution reasoning, and debugging, whereas IP is particularly prominent in generation and translation. The raw-to-collapsed length ratio remains relatively stable across tasks (1.3–1.7), indicating a comparable degree of consecutive action repetition. Thus, task-specific structures emerge primarily after the shared problem-understanding stage.

Cross-Model Comparison. The two R1 models exhibit highly consistent task-level reasoning structures but differ in how these structures are realized. Across all four tasks, R1-Qwen3-8B generates longer reasoning traces than R1-0528 (+37.1% for code generation, +133.1% for code execution reasoning, +21.8% for program debugging, and +7.6% for code translation) and consistently exhibits equal or higher transition entropy, particularly for execution reasoning and debugging. Thus, the two models exhibit similar task-level structures, while R1-Qwen3-8B tends to realize them through longer and more dispersed trajectories.

Finding 1. Across the two R1 models, each coding task exhibits a distinct general reasoning pattern that reflects its underlying problem-solving strategy. Code generation follows a design-to-implementation pattern, code execution reasoning relies on state tracing, program debugging follows an evidence-fix-recheck cycle, and code translation is driven by semantic mapping followed by implementation.

4 RQ2: Positive and Negative Patterns

4.1 Setup

RQ2 characterizes action-level reasoning differences between valid and invalid reasoning traces using the positive and negative pattern mining procedure described in Section 2.3.3. We analyze 2,452 valid and 924 invalid traces generated by the two primary R1 models across four coding tasks. A pattern is retained when its support in the enriched group exceeds 0.08 and its support difference from the comparison group exceeds 0.06. These thresholds were calibrated by two domain experts through empirical inspection of preliminary results.

4.2 Results

We first examine differences between valid and invalid reasoning action traces, and then present the resulting positive and negative patterns that characterize effective and ineffective reasoning trajectories. These analyses reveal differences in fine-grained action dynamics between valid and invalid reasoning traces.

Invalid Traces Shift Action Allocation Toward Drift-oriented Actions. Table 3 summarizes reasoning-token counts, action-trace lengths, and action-allocation statistics for valid and invalid traces across the four oracle-backed tasks. For both models, invalid traces are consistently longer than valid traces in reasoning tokens, raw action length, and collapsed action length, indicating greater generated reasoning volume and more action steps in invalid traces.

The two groups also differ substantially in action allocation. Invalid traces exhibit lower closure shares (R1-0528: 0.414 $\rightarrow$ 0.241; R1-Qwen3-8B: 0.330 $\rightarrow$ 0.219) and higher drift shares (R1-0528:

Table 3. Length and Action Allocation of Valid and Invalid Reasoning Traces

Metric	R1-0528		R1-Qwen3-8B	
	Valid	Invalid	Valid	Invalid
Reasoning tokens	3,690	6,609	4,330	6,991
Raw action length	18.58	24.96	22.64	32.21
Collapsed action length	11.78	14.91	15.97	21.99
Closure share	0.414	0.241	0.330	0.219
Drift share	0.247	0.391	0.313	0.464
Closure–Drift	+0.167	−0.150	+0.017	−0.245

0.247 $\rightarrow$ 0.391; R1-Qwen3-8B: 0.313 $\rightarrow$ 0.464). This shift corresponds to less allocation to verification, code construction, and auxiliary actions and greater allocation to solution design, meta-reasoning, and knowledge retrieval.

The closure-minus-drift score further summarizes this difference. Its value changes from positive to negative between valid and invalid traces for both models (R1-0528: +0.167 $\rightarrow$ −0.150; R1-Qwen3-8B: +0.017 $\rightarrow$ −0.245), indicating that invalid traces allocate relatively more reasoning to drift-oriented than closure-oriented actions.

Positive and Negative Patterns Distinguish Convergent Iteration from Repeated Reopening. We next examine whether the differences in action allocation are reflected in local reasoning structures. Table 4 summarizes the discovered positive and negative patterns, while Figure 2 visualizes the support contrasts of representative high-support patterns across task–model settings. The results show that valid and invalid traces exhibit distinct local transition patterns. For example, for R1-Qwen3-8B on code generation, the positive pattern $VV \rightarrow IP \rightarrow VV$ occurs in 46.6% of valid traces, compared with 19.6% of invalid traces. In contrast, the negative pattern $KR \leftrightarrow SD$ occurs in 56.4% of invalid traces but only 15.9% of valid traces. A similar contrast appears in debugging for R1-0528: $CC \rightarrow VV \rightarrow AUX$ occurs in 21.7% of valid traces versus 4.0% of invalid traces, whereas $VV \rightarrow SD \rightarrow MR$ occurs in 20.0% of invalid traces versus 2.7% of valid traces.

These patterns indicate two qualitatively different forms of iteration. Positive patterns are predominantly characterized by transitions that return to verification after planning or implementation, such as $VV \rightarrow IP \rightarrow VV$, or that proceed from implementation to verification and auxiliary completion, such as $CC \rightarrow VV \rightarrow AUX$. Negative patterns, in contrast, more often involve repeated transitions among design, reflection, and knowledge retrieval, including cycles such as $KR \leftrightarrow SD$. Thus, the distinction between valid and invalid traces is reflected not only in the overall allocation of reasoning actions but also in how these actions are locally sequenced.

We further assess the representative support contrasts using two-sided Fisher’s exact tests with Benjamini–Hochberg correction. The complete support, count, p -value, and q -value results are provided in the replication package. These results indicate that the relationship between local iteration patterns and trace validity depends on the specific actions involved and their transition structure, rather than on iteration frequency alone.

Cross-model Consistency with Task-dependent Strength. Finally, we examine the consistency of the observed validity contrasts across models and coding tasks. The distinction between closure-oriented and drift-oriented reasoning is observed in both primary R1 models, suggesting that the identified differences are not specific to a single model within this model family. The same direction of contrast is also reflected in the local patterns: valid traces more frequently contain

Table 4. Positive and Negative Patterns Identified across Four Coding Tasks. Gray-shaded cells denote negative patterns.

Task	Model	Positive/Negative Pattern	Representative Signature
Generation	R1-0528	Plan-verify-close	IP→VV→AUX
		Plan-verify iteration	IP→VV→IP→VV→IP
		Knowledge-verify-reflect detour	KR→VV→MR
		Design-knowledge detour	SD→KR→IP
	R1-Qwen3-8B	Verify-plan-reverify	VV→IP→VV
		Extended verify-plan	VV→IP→VV→IP
		Knowledge-design loop	KR↔SD
		Reflection-knowledge loop	MR↔KR
Execution Reasoning	R1-0528	State-trace verification	PU→VV→MR→VV
		Understand-reopening reflection	MR→PU→MR
	R1-Qwen3-8B	State-trace verification	PU→VV→MR→VV
		Reflection-design reopening	MR→SD→{PU,VV}
Debugging	R1-0528	Change-verify-close	CC→VV→AUX
		Verify-design-reflect reopening	VV→SD→MR
		Design-reflect detour	SD→MR→IP
	R1-Qwen3-8B	Verify-plan-reverify	VV→IP→VV
Translation	R1-0528	Design-plan-construct	SD→IP→CC
		Design-reflect reopening	SD→MR→SD→MR→SD
		Verify-plan-construct detour	VV→IP→CC
	R1-Qwen3-8B	Understand-design-plan-reflect	PU→SD→IP→MR
		Design-reflect replanning	SD→MR→SD→IP
		Understand-design reopening	PU→SD→PU→SD

verification-centered transitions, whereas invalid traces more frequently contain repeated transitions involving design revision, reflection, and knowledge retrieval.

The magnitude of these differences, however, varies across settings. The contrast is more pronounced for R1-Qwen3-8B, whose invalid traces exhibit higher drift shares and more frequent cycles involving *SD*, *MR*, and *KR*. This difference is also reflected in trace length. For code generation, invalid traces contain substantially more reasoning tokens than valid traces for both R1-0528 (17,163 vs. 7,102) and R1-Qwen3-8B (14,925 vs. 8,598), with a stronger concentration of drift-oriented actions. Code execution reasoning and debugging exhibit similar but more structured contrasts, whereas the differences are smaller and less symmetric for code translation.

Finding 2. Valid and invalid reasoning traces differ in trace length, action allocation, and local reasoning patterns. Positive patterns in valid traces predominantly reflect verification-centered iteration, whereas negative patterns in invalid traces more often involve repeated cycles of design revision, reflection, and knowledge retrieval.

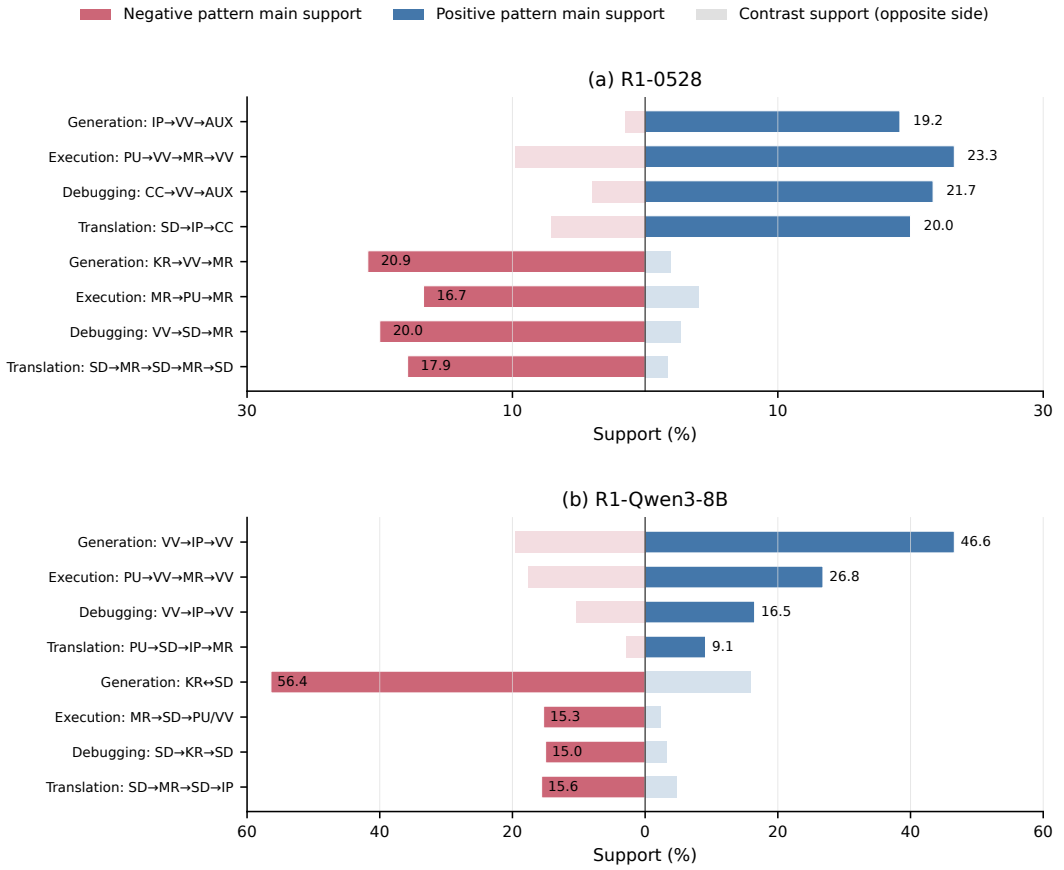


Fig. 2. Support Comparison between Positive and Negative Patterns.

5 RQ3: Applications of Reasoning Patterns

To examine how the discovered reasoning patterns can be leveraged at inference time, we evaluate three applications covering task identification, pattern-guided prompting, and pattern-aware early stopping. Specifically, we investigate:

- **RQ3.1** Can general reasoning patterns support reliable identification of different AI coding tasks? (Section 5.1)
- **RQ3.2** How does prompting guided by positive and negative patterns affect coding performance? (Section 5.2)
- **RQ3.3** How does negative pattern-aware early stopping affect coding accuracy and token usage? (Section 5.3)

5.1 Task Identification from General-Pattern Features [shen: add the setup and results about transferring to unseen models.]

Setup. Following the task-identification probe described in Section 2.4, RQ3.1 examines whether coding tasks can be identified using general-pattern features derived solely from action traces. The evaluation dataset contains 3,376 reasoning traces generated by two models across four coding tasks. We evaluate two standard classifiers, logistic regression and random forest, using 5-fold

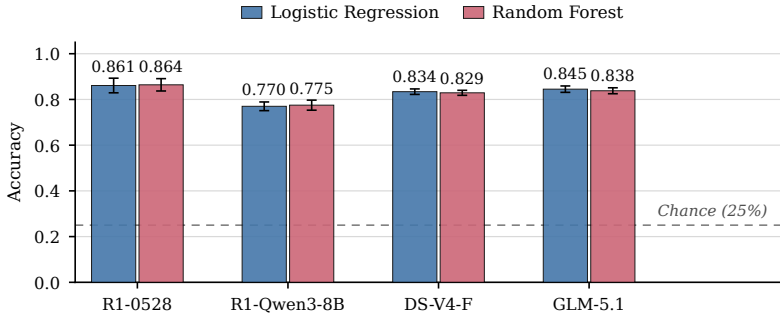


Fig. 3. Task Identification Accuracy Using General-Pattern Features. Error bars denote the standard deviation across cross-validation folds, and the dashed line indicates the random baseline of 25%. [shen: explain the transfer....]

stratified cross-validation. We report classification accuracy as the mean and standard deviation across folds; for the four-class setting, random guessing yields an accuracy of 25%.

Results. Using 57 general-pattern features, logistic regression achieves an accuracy of 0.861 ± 0.032 for R1-0528 and 0.770 ± 0.019 for R1-Qwen3-8B. Random forest yields comparable performance, with accuracies of 0.864 ± 0.027 and 0.775 ± 0.022 , respectively. Both classifiers substantially outperform the 25% random baseline, indicating that general-pattern features provide discriminative signals for distinguishing the four coding tasks across both models. Importantly, the classifiers operate solely on features derived from reasoning action traces and have no access to task descriptions, final answers, or external metadata. The observed performance therefore indicates that task identity is reflected in the structural organization of model reasoning traces.

To further examine which aspects of the reasoning traces contribute to task identification, we analyze feature importance using mutual information. For both models, the most informative features primarily capture the allocation of reasoning effort across core actions, including the design ratio and the distributions of *PU*, *SD*, *IP*, and *VV*. These results suggest that action allocation provides a particularly informative signal for distinguishing coding tasks.

Finding 3.1. General-pattern features derived solely from reasoning action traces support accurate identification of coding tasks, with both classifiers substantially outperforming the random baseline. Action-allocation features provide the most informative signals, indicating that task identity is reflected in the structural organization of LRM reasoning traces.

5.2 Pattern-guided Prompting

Setup. RQ3.2 evaluates whether positive and negative reasoning patterns can be used as inference-time guidance for coding tasks. We compare two prompting conditions: a *default prompt*, which contains the original task instruction without additional reasoning guidance, and a *pattern-guided prompt*, which augments the same instruction with task-specific guidance derived from the mined positive and negative patterns. All other task inputs and prompting settings are kept unchanged between the two conditions.

For the direct setting, we apply each primary R1 model's own pattern inventory to the corresponding model across the four coding tasks with objective correctness oracles. To evaluate cross-model transfer, we apply the pattern inventories mined from the R1 models to two previously unseen models from different model families, DeepSeek-V4-Flash (DS-V4-F) [8] and GLM-5.1 [37].

Table 5. Performance and Cross-Model Transfer of Pattern-Guided Prompting. ****. [shen: explain the Direct and Transferred.]

Task	Method	Direct				Transferred			
		R1-0528		R1-Qwen3-8B		DS-V4-F		GLM-5.1	
		P@1↑	Tok.↓	P@1↑	Tok.↓	P@1↑	Tok.↓	P@1↑	Tok.↓
Generation	Default	72.46	14.9	56.21	17.9	80.78	13.0	90.29	16.5
	Positive Patterns	73.14	15.2	55.86	18.3	84.36	13.5	90.06	13.2
	Negative Patterns	72.08	14.6	56.39	17.6	79.84	12.5	90.74	13.0
	Positive and Negative Patterns	74.49	15.6	56.88	17.3	87.80	12.2	91.17	12.9
Execution Reasoning	Default	55.76	1.7	44.24	3.4	56.81	2.1	59.84	1.6
	Positive Patterns	57.16	2.4	48.17	4.4	58.21	2.6	60.32	1.6
	Negative Patterns	56.63	2.1	46.86	3.9	57.54	2.3	60.41	1.5
	Positive and Negative Patterns	58.37	2.9	49.74	4.7	59.27	2.8	60.73	1.5
Debugging	Default	88.80	4.5	61.20	7.7	91.40	2.4	93.00	2.8
	Positive Patterns	91.20	5.8	61.54	7.8	90.92	2.5	93.02	2.6
	Negative Patterns	90.47	5.1	61.08	7.6	92.18	2.2	93.11	2.6
	Positive and Negative Patterns	94.60	6.6	61.72	7.9	92.80	2.3	93.20	2.5
Translation	Default	55.08	5.7	39.09	4.2	55.05	2.1	56.97	2.5
	Positive Patterns	56.81	6.5	42.51	4.4	54.73	2.2	58.03	2.4
	Negative Patterns	57.44	6.2	43.27	4.3	56.73	1.8	58.52	2.4
	Positive and Negative Patterns	58.78	7.3	44.14	4.5	57.44	1.9	58.75	2.3

This design allows us to examine both within-model effectiveness and transferability of pattern-guided prompting.

We evaluate coding performance using pass@1 [3]. To reduce variance from stochastic decoding, each condition is evaluated in three independent runs, and we report the mean pass@1 across runs. We additionally report the mean number of generated output tokens per instance, measured in thousands, to characterize the effect of prompting on output length.

Results. Table 5 summarizes the effectiveness of prompting guided by positive and negative patterns across four AI coding tasks.

1) *Direct Results.* Pattern-guided prompting improves mean pass@1 on all four tasks for both R1-0528 and R1-Qwen3-8B, with the largest gains on debugging for R1-0528 and on execution reasoning and translation for R1-Qwen3-8B (+5.80, +5.50, and +5.05 points, respectively).

Improvement and token usage are task-dependent. For example, R1-0528 debugging improves by 5.80 percentage points while generated-output tokens increase from 4.5K to 6.6K; R1-Qwen3-8B generation improves by 0.67 points while tokens decrease from 17.9K to 17.3K. These output-token counts exclude the additional prompt tokens.

2) *Ablation: Positive-Pattern Guidance vs. Negative-Pattern Constraints.* To isolate the contribution of each guidance component, we further evaluate two ablated variants: *Positive-only*, which retains only the positive patterns mined from valid traces, and *Negative-only*, which retains only the negative constraints derived from negative patterns. Across the sixteen task-model settings, each component alone yields inconsistent gains and can even underperform the default prompt: Positive-only degrades R1-Qwen3-8B generation (55.86 vs. 56.21) and DS-V4-F translation (54.73 vs. 55.05), while Negative-only degrades DS-V4-F generation (79.84 vs. 80.78) and R1-Qwen3-8B debugging (61.08 vs. 61.20). In contrast, their combination achieves the highest pass@1 among all guided variants in every task-model setting and surpasses the default prompt in all sixteen settings. The observed combination effect is consistent with complementary roles for positive and

negative pattern constraints: one may provide a direction toward closure, while the other discourages reopening loops. The current ablations do not establish this mechanism or distinguish pattern-specific effects from generic advice.

3) *Transferred Results*. We next assess the transfer of R1-family guidance to the two additional target models. On DS-V4-F, pass@1 improves on all four tasks, with the largest gain on code generation (+7.02 points); token use increases on execution reasoning and decreases on the other three tasks. On GLM-5.1, pass@1 improves on all four tasks by 0.20–1.78 percentage points, while token use decreases on all four tasks. For generation, pass@1 increases from 90.29 to 91.17 (+0.88 points) and token use falls from 16.5K to 12.9K. Thus, guidance improves mean pass@1 across both transfer models, although the magnitude of improvement and token usage vary by setting.

Finding 3.2. Pattern-guided prompts combining positive and negative pattern constraints improve mean pass@1 in all 16 task–model settings, including transfer to two additional models. The combined guidance achieves higher mean pass@1 than either component alone in all evaluated settings, suggesting complementary benefits from the two components.

5.3 Negative Pattern-aware Early Stopping

Setup. RQ3.3 examines whether negative pattern signals can be leveraged for online early stopping to change the accuracy–token trade-off. We compare two settings: *Full Reasoning*, which allows the model to complete its full reasoning process before producing an answer, and *Pattern-aware Stop*, an online stopping policy defined in Section 2.4 that monitors the streaming reasoning trace and triggers answer finalization when a drift, closure, or repeated-text condition is met. Since Pattern-aware Stop requires access to streaming reasoning outputs, we apply it to R1-Qwen3-8B and two additional target models, DS-V4-F and GLM-5.1, while R1-0528 is excluded due to the lack of a streaming-capable interface during evaluation. Following RQ3.2, each setting is executed three times, and we report mean pass@1 together with average per-instance API-reported token usage.

The Tok. metric is taken from the same target-model API usage field for both settings and includes finalizer output for stopped runs. The local rule-based monitor makes no separate annotation-model call, whereas the second request to the target model resends the task and stopped reasoning prefix. Tok. measures API-reported token usage rather than elapsed time or monetary cost.

In an offline replay of 40 stored R1-Qwen3-8B traces split into 9,019 25-character chunks, local monitor processing took 0.13 ms per chunk at the median (0.30 ms at the 95th percentile); cumulative local CPU time was 19 ms per trace at the median (142 ms at the 95th percentile). This microbenchmark excludes model, network, and finalizer latency.

Table 6. Pattern-aware Early Stopping Performance across Models. Tok.: API-reported token usage (thousands); stopped runs include finalizer output.

Task	Model	Full Reasoning		Pattern-aware Stop			
		P@1↑	Tok.↓	P@1↑	Tok.↓	$\Delta P@1$ (pp)	$\Delta Tok.$ (%)
Generation	R1-Qwen3-8B	56.21	17.89K	52.14	10.64K	-4.07	-40.56%
	DS-V4-F	80.78	13.00K	73.36	8.43K	-7.42	-35.13%
	GLM-5.1	90.29	16.54K	88.06	7.85K	-2.23	-52.52%
Execution Reasoning	R1-Qwen3-8B	44.24	3.43K	40.58	2.10K	-3.66	-38.75%
	DS-V4-F	56.81	2.13K	51.57	0.85K	-5.24	-59.78%
	GLM-5.1	59.84	1.64K	58.63	0.52K	-1.21	-68.53%
Debugging	R1-Qwen3-8B	61.20	7.65K	54.40	4.18K	-6.80	-45.32%
	DS-V4-F	91.40	2.42K	86.60	1.69K	-4.80	-30.16%
	GLM-5.1	93.00	2.77K	91.20	1.74K	-1.80	-37.16%
Translation	R1-Qwen3-8B	39.09	4.18K	37.64	2.33K	-1.45	-44.22%
	DS-V4-F	55.05	2.14K	50.79	1.23K	-4.26	-42.65%
	GLM-5.1	56.97	2.53K	55.48	1.51K	-1.49	-40.56%

Results. Table 6 reports the RQ3.3 comparison across four coding tasks. For R1-Qwen3-8B, reported token usage decreases by 38.75%–45.32%, while pass@1 decreases by 1.45–6.80 percentage points. Translation has the smallest accuracy decrease (1.45 points) at a 44.22% token reduction; generation and debugging have larger accuracy losses.

Reported token usage also decreases on the two additional models, DS-V4-F (30.16%–59.78%) and GLM-5.1 (37.16%–68.53%). Their pass@1 decreases by 4.26–7.42 and 1.21–2.23 points, respectively. These results show an accuracy–token trade-off across the evaluated models.

Finding 3.3. Pattern-aware early stopping reduces API-reported token usage by 30–69%, with a 1.21–7.42 percentage-point decrease in pass@1 across the three evaluated models, demonstrating an accuracy–token trade-off across task–model settings.

6 Discussion

6.1 Understanding Reasoning Structures

Our results suggest that reasoning in AI coding is organized around recurrent action-level structures rather than isolated step sequences or linear chains of thought. Across tasks, we observe stable general behavioral regularities, indicating that reasoning traces exhibit compositional structure that is consistent within tasks but distinct across them.

At a finer granularity, the contrast between valid and invalid traces suggests that reasoning quality is closely related to how actions evolve toward convergence. This is reflected in the identified positive and negative patterns, which capture differences in action-level progression between traces judged valid and invalid.

6.2 Future Research Directions

Our findings suggest several promising directions for future research on reasoning in AI coding.

Evaluation of Reasoning Models. Reasoning traces provide structured behavioral signals that are not captured by outcome-based metrics such as pass@1 or trace length. This motivates future work on evaluation frameworks grounded in action-level reasoning structure, enabling finer-grained assessment of model reasoning behavior.

Structure-Aware Inference Control. The discovered reasoning patterns suggest the potential for controlling inference through structural signals, enabling fine-grained monitoring and intervention during reasoning, including but not limited to early stopping.

Learning with Reasoning Structure. Beyond pattern-guided prompting, pattern-augmented reasoning traces expose intermediate structures that may serve as supervision signals for training models that explicitly optimize reasoning processes.

7 Threats to Validity

Construct Validity. Our analysis abstracts free-form reasoning traces into discrete reasoning actions, enabling systematic comparison but potentially losing fine-grained semantic distinctions and internal reasoning nuances. We mitigate this limitation by grounding the taxonomy in prior work, refining it via expert inspection, and validating automatic annotations against human re-labeling (Cohen’s $\kappa = 0.816$). However, alternative taxonomies, segmentation granularities, or collapsing rules may lead to different patterns, suggesting the need for sensitivity analysis in future work.

Internal Validity. Trace-validity labels are derived from an LLM-based judge ensemble, with human adjudication of low-confidence cases. Confidence-weighted voting reduces judgment variance, but these labels may still miss subtle reasoning errors. Task-level functional correctness is evaluated separately using objective oracles and summarized as pass@1. Nonetheless, prompting and early-stopping results should be interpreted as intervention evidence rather than causal attribution to specific mined patterns, motivating future controlled interventions for causal validation. For early stopping, the online rule-based mapper is distinct from the offline LLM action annotators, and their agreement on partial prefixes has not yet been measured. Mapping errors may therefore alter motif detection and stopping decisions. The stopping policy also includes a repeated-text trigger independent of action motifs.

External Validity. Our study spans four coding tasks and four model configurations, which may not fully represent the diversity of software engineering scenarios or model architectures. R1-0528 and its Qwen3-8B distillation share a lineage, and DS-V4-F is another DeepSeek model; GLM-5.1 provides one external model-family test. Further validation across tasks, languages, and model families is required to assess generality. GLM-5.1 is an additional target model in RQ3; it also participated in the validity-judge ensemble for the primary R1 traces. Its intervention results thus provide cross-family evidence, but they are not a fully independent replication of the entire discovery-to-intervention pipeline.

8 Related Work

Recent LRMs incorporate extended reasoning traces as a core component of inference [6]. In code-related settings, prior work shows that reasoning-before-coding is not uniformly beneficial and depends on how reasoning is structured relative to code generation [18]. Other studies explore leveraging external or structured signals to improve reasoning, such as mining discussion data into training chains [34] or validating intermediate steps via search-based refinement [28]. These efforts demonstrate the utility of explicit reasoning for code generation, but primarily focus on improving outputs or training signals rather than characterizing recurring reasoning structures across tasks.

Prior work also examines reasoning quality and failure modes. Studies show that reasoning quality is not reliably aligned with final correctness, as flawed reasoning may still produce correct outputs and vice versa [39], and that superficial signals such as step count or verbosity are weak indicators of correctness [31]. Recent work further categorizes reasoning behaviors and failure patterns such as hallucination, overthinking, and erroneous backtracking [1, 12, 23]. While these studies move beyond outcome-only evaluation, they are often limited to single tasks or rely on coarse-grained or manual annotations, leaving open how reasoning structures vary across tasks and how they differ between valid and invalid traces.

Reasoning traces have also been studied for inference-time control. Methods such as SEER adapt reasoning strategies during generation [10], while work on overthinking motivates compressing redundant reasoning steps [4, 14, 38]. Early-stopping approaches exploit intermediate signals to terminate generation based on convergence or confidence criteria [19, 33]. These methods primarily optimize efficiency or decoding behavior, but do not explicitly derive reasoning patterns from empirical trace analysis.

In contrast, we treat reasoning traces as empirical behavioral artifacts, abstracting them into reasoning action traces and analyzing recurring structures across code generation, code execution reasoning, program debugging, and code translation. This enables a unified characterization of general reasoning patterns and validity-associated positive and negative patterns, and further connects these structures to downstream applications in task identification, pattern-guided prompting, and pattern-aware early stopping.

9 Conclusion

In this paper, we present an empirical study of reasoning patterns in AI coding. By abstracting reasoning traces into action sequences, we identify general patterns that capture task-specific reasoning strategies and positive and negative patterns that characterize valid and invalid reasoning behaviors at the action level across four coding tasks. We further demonstrate the utility of these patterns for inference-time analysis and control through task identification, pattern-guided prompting, and negative pattern-aware early stopping. These results show that reasoning patterns can provide a structured basis for analyzing LRM reasoning and controlling inference-time behavior in AI coding.

Data Availability

All code and data used in this study are publicly available at: https://anonymous.4open.science/r/FSE_2027. The repository includes: (i) the collected reasoning traces and annotated datasets across the four tasks (code generation, code execution reasoning, debugging, and translation); (ii) the code for pattern discovery, pattern-guided prompting and pattern-aware early stopping (RQ1–RQ3); and (iii) all experimental results and evaluation scripts.

References

- [1] Mohammad Abdollahi, Khandaker Rifah Tasnia, Soumit Kanti Saha, Jinqiu Yang, Song Wang, and Hadi Hemmati. 2025. Demystifying errors in LLM reasoning traces: An empirical study of code execution simulation. *ACM Transactions on Software Engineering and Methodology* (2025).
- [2] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57, 1 (1995), 289–300.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [4] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. [n. d.]. Do NOT Think That Much for $2+3=?$

- On the Overthinking of Long Reasoning Models. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research)*. PMLR / OpenReview.net.
- [5] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [6] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR* abs/2501.12948 (2025).
- [7] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *CoRR* abs/2512.02556 (2025).
- [8] DeepSeek-AI. 2026. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. Technical Report. DeepSeek-AI. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf
- [9] Pengfei Gao and Chao Peng. [n. d.]. More with Less: An Empirical Study of Turn-Control Strategies for Efficient Coding Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12-18, 2026, Rio de Janeiro, Brazil*.
- [10] Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, and Michael R. Lyu. 2025. SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning. *CoRR* abs/2510.17130 (2025).
- [11] Google DeepMind. 2025. Gemini 3 Flash Model Card. <https://deepmind.google/models/model-cards/gemini-3-flash/>. Accessed: 2026-05-28.
- [12] Kevin Halim, Sin G. Teo, Ruitao Feng, Zhenpeng Chen, Yang Gu, Chong Wang, and Yang Liu. 2025. A Study on Thinking Patterns of Large Reasoning Models in Code Generation. *CoRR* abs/2509.13758 (2025).
- [13] Dong Huang, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Codcot: Tackling code syntax errors in CoT reasoning for code generation. *CoRR* abs/2308.08784 (2023).
- [14] Kerui Huang, Shuhan Liu, Xing Hu, Tongtong Xu, Lingfeng Bao, and Xin Xia. 2025. Reasoning Efficiently Through Adaptive Chain-of-Thought Compression: A Self-Optimizing Framework. *CoRR* abs/2509.14093 (2025).
- [15] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. [n. d.]. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*.
- [16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. [n. d.]. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- [17] Qintong Li, Leyang Cui, Lingpeng Kong, and Wei Bi. 2025. Exploring the Reliability of Large Language Models as Customized Evaluators for Diverse NLP Tasks. In *Proceedings of the 31st International Conference on Computational Linguistics*. Association for Computational Linguistics, Abu Dhabi, UAE, 10325–10344.
- [18] Renbiao Liu, Anqi Li, Chaoding Yang, Hui Sun, and Ming Li. [n. d.]. Revisiting Chain-of-Thought in Code Generation: Do Language Models Need to Learn Reasoning before Coding?. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. PMLR / OpenReview.net.
- [19] Xin Liu and Lu Wang. [n. d.]. Answer Convergence as a Signal for Early Stopping in Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025*. Association for Computational Linguistics, 17896–17907. <https://aclanthology.org/2025.emnlp-main.904>
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. [n. d.]. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- [21] Monoshi Kumar Roy, Simin Chen, Benjamin Steenhoeck, Jinjun Peng, Gail E. Kaiser, Baishakhi Ray, and Wei Le. [n. d.]. CodeSense: a Real-World Benchmark and Dataset for Code Semantic Reasoning. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*. OpenReview.net. <https://openreview.net/forum?id=ehXVDJm0PS>
- [22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. [n. d.]. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- [23] Zhongxiang Sun, Qipeng Wang, Haoyu Wang, Xiao Zhang, and Jun Xu. 2025. Detection and Mitigation of Hallucination in Large Reasoning Models: A Mechanistic Perspective. *CoRR* abs/2505.12886 (2025).
- [24] Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *CoRR* abs/2507.06261 (2025).
- [25] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Hui, Weichuan Liu, Zhiyuan Liu, and Maosong Sun. [n. d.]. DebugBench: Evaluating Debugging Capability of Large Language Models. In *Findings*

- of the Association for Computational Linguistics, *ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*. 4173–4198.
- [26] Xincheng Wang, Ruida Hu, Pengfei Gao, Chao Peng, and Cuiyun Gao. [n. d.]. An Agent-based Evaluation Framework for Complex Code Generation. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 2427–2439.
- [27] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. [n. d.]. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.
- [28] Yibo Wang, Zhihao Peng, Ying Wang, Zhao Wei, Hai Yu, and Zhiliang Zhu. [n. d.]. MCTS-Refined CoT: High-Quality Fine-Tuning Data for LLM-Based Repository Issue Resolution. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 1844–1855.
- [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. [n. d.]. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- [30] Qiujie Xie, Qingqiu Li, Zhuohao Yu, Yuejie Zhang, Yue Zhang, and Linyi Yang. [n. d.]. An Empirical Analysis of Uncertainty in Large Language Model Evaluations. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- [31] Haoran Xue, Gias Uddin, and Song Wang. [n. d.]. An Empirical Study of Reasoning Steps in Thinking Code LLMs. In *3rd ACM International Conference on AI-powered Software, AIware 2026*. <https://openreview.net/forum?id=2WgHyQZ2iO>
- [32] Pengyu Xue, Linhao Wu, Zhen Yang, Chengyi Wang, Xiang Li, Yuxiang Zhang, Jia Li, Ruikai Jin, Yifei Pei, Zhaoyan Shen, Xiran Lyu, and Jacky Wai Keung. 2025. ClassEval-T: Evaluating Large Language Models in Class-Level Code Translation. *Proc. ACM Softw. Eng.* 2, ISSTA (2025), 1421–1444.
- [33] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. [n. d.]. Dynamic Early Exit in Reasoning Models. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*. <https://arxiv.org/abs/2504.15895>
- [34] Chengran Yang, Zhensu Sun, Hong Jin Kang, Jieke Shi, and David Lo. [n. d.]. Think Like Human Developers: Harnessing Community Knowledge for Structured Code Reasoning. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12-18, 2026, Rio de Janeiro, Brazil*.
- [35] Dayu Yang, Tianyang Liu, Daoan Zhang, Antoine Simoulin, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Xin Qian, Grey Yang, Jiebo Luo, et al. 2025. Code to think, think to code: A survey on code-enhanced reasoning and reasoning-driven code intelligence in llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2586–2616.
- [36] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. [n. d.]. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- [37] Z.ai. 2026. GLM-5.1 Model Card. <https://huggingface.co/zai-org/GLM-5.1>. Accessed: 2026-06-15.
- [38] Wenhao Zeng, Yaoning Wang, Chao Hu, Yuling Shi, Chengcheng Wan, Hongyu Zhang, and Xiaodong Gu. 2025. Pruning the Unsurprising: Efficient Code Reasoning via First-Token Surprisal. *CoRR* abs/2508.05988 (2025).
- [39] Binqun Zhang, Li Zhang, Zhiwen Luo, Yuxin Du, Fang Liu, Song Wang, and Lin Shi. 2026. Are They All Good? Evaluating the Quality of CoTs in LLM-based Code Generation. *IEEE Transactions on Software Engineering* (2026), 1928–1943. doi:10.1109/TSE.2026.3676295
- [40] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. [n. d.]. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.